

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский поли-
технический университет»**

Электротехнический факультет

Кафедра «Информационные технологии и автоматизированные системы»
направление подготовки: 09.03.01– «Информатика и вычислительная тех-
ника»

**Отчёт по практической работе на тему
«Разработка автоматизированного рабочего места специалиста
по учёту продуктов»**

Выполнил студент гр. ИВТ-25-16

Еговцев Иван Сергеевич

Проверил:

(оценка)

(подпись)

(дата)

г. Пермь, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 АНАЛИЗ ТРЕБОВАНИЙ И ПРОЕКТИРОВАНИЕ	4
1.1 Основные требования.....	4
1.2 Проектирование	4
1.2.1 Классы.....	4
1.2.1.1 Класс Parser	5
1.2.1.2 Класс FixedLineEdit.....	5
1.2.1.3 Класс Window	7
1.2.2 Модуль prods.cpp	7
1.2.3 Модуль recipes.cpp.....	8
1.2.4 Модуль window.cpp	9
2 ИСПОЛЬЗУЕМЫЕ ИНСТРУМЕНТЫ.....	10
2.1 Используемые библиотеки	10
2.1.1 Фреймворк Qt.....	10
2.1.2 Библиотека plohmam/json.....	10
2.1.3 Стандартная библиотека C++	10
2.2 Другие инструменты	10
ЗАКЛЮЧЕНИЕ	12
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	14

ВВЕДЕНИЕ

Актуальность работы:

В современном мире люди всё чаще сталкиваются с необходимостью контролировать пищевые аллергены и сроки годности продуктов. Ведение учёта вручную (блокноты, таблицы) неудобно и чревато ошибками. Разработанное автоматизированное рабочее место (АРМ) позволяет быстро добавлять продукты по штрихкоду, отслеживать остатки, контролировать аллергены, получать предупреждение о просрочке и автоматически определять доступность блюд по рецептам. Программа упрощает управление домашними запасами и снижает риск употребления опасных или испорченных продуктов.

Цель работы:

Целью проекта является создание приложения для домашнего учёта продуктов питания с функциями контроля аллергенов, отслеживания сроков годности и автоматического определения возможности приготовления блюд по рецептам.

Задачи:

- 1) Определить, какие библиотеки будут использоваться в проекте.
- 2) Продумать структуру проекта.
- 3) Создать функции ввода продуктов по штрихкоду (ручной и автоматический).
- 4) Создать функции управления запасами и рецептами, контроль сроков годности продуктов.
- 5) Написать функции отслеживания аллергенов и доступных рецептов.
- 6) Реализовать окно программы с помощью фреймворка Qt [1].
- 7) Протестировать работу проекта.
- 8) Записать видео работы программы.
- 9) Составить отчёт о проделанной работе.

1 АНАЛИЗ ТРЕБОВАНИЙ И ПРОЕКТИРОВАНИЕ

1.1 Основные требования

Перед непосредственной разработкой были определены основные требования к работе программы:

- Пользователь вводит продукт или изменяет его путём ввода его штрихкода.
- Длина вводимого штрихкода строго 13 символов;
- Программа хранит следующую информацию о продукте:
 - название;
 - производитель;
 - объём одной единицы товара;
 - количество;
 - суммарный объём;
 - аллергены, содержащиеся в продукте;
 - срок годности после вскрытия;
 - дата вскрытия продукта, если такая функция включена.
- Информацию о продукте программа сначала пытается получить с сайта. Если с сайта получить информацию не удалось, предложить пользователю ручной ввод.
- Отслеживать срок годности продукта программа начинает только тогда, когда эта опция включена для продукта и его общий объём не кратен объёму одной единицы.
- Все файлы, с которыми работает программа, имеют расширение *.json, для удобства и наглядности работы.

1.2 Проектирование

1.2.1 Классы

В программе используется 3 пользовательских класса:

- Parser;
- Window;

- FixedLineEdit.

1.2.1.1 Класс Parser

Этот класс обеспечивает взаимодействие с внешним API (Open Food Facts) для получения информации о продукте по его штрихкоду. Выполняет синхронный HTTP-запрос, выгружает JSON-ответ и извлекает название, бренд, объём и список аллергенов. Для удобства UML диаграмма класса представлена на рисунке 1.

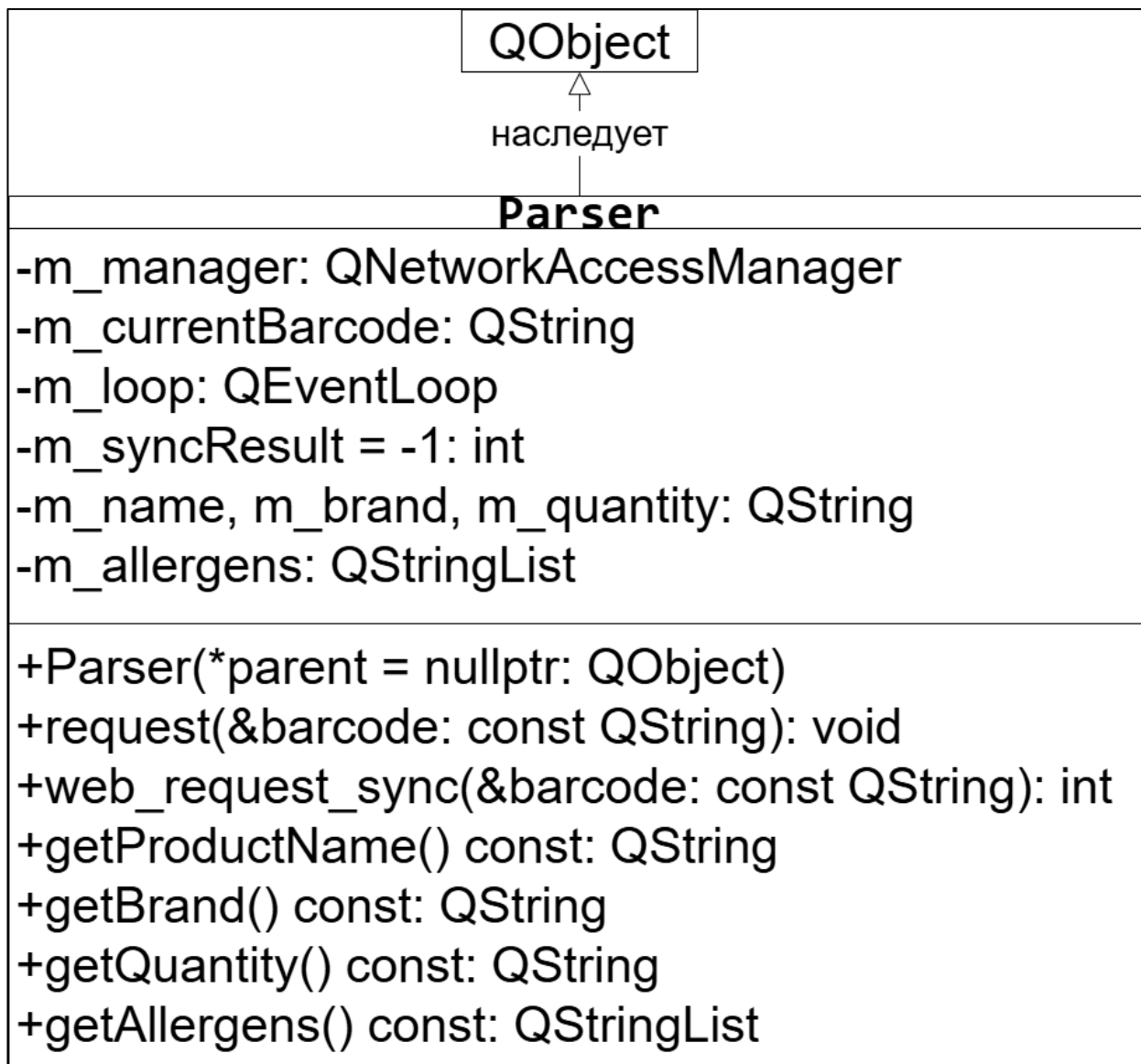


Рисунок 1 – UML диаграмма класса Parser

1.2.1.2 Класс FixedLineEdit

FixedLineEdit предоставляет специализированное поле ввода для штрихкода, которое проверяет вводимую строку на соответствие формату (13 цифр)

и визуально сигнализирует о валидности (специальным сообщением). Генерирует сигнал validationResult при завершении ввода (по нажатию Enter или при потере фокуса). Диаграмма класса представлена на рисунке 2.

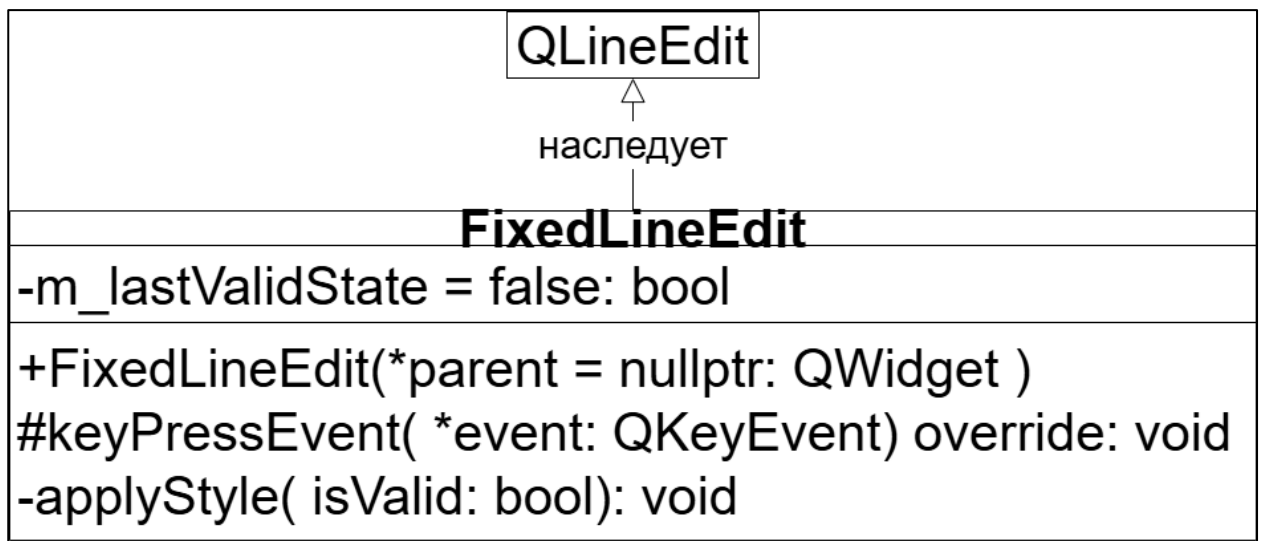


Рисунок 2 – UML диаграмма класса FixedLineEdit

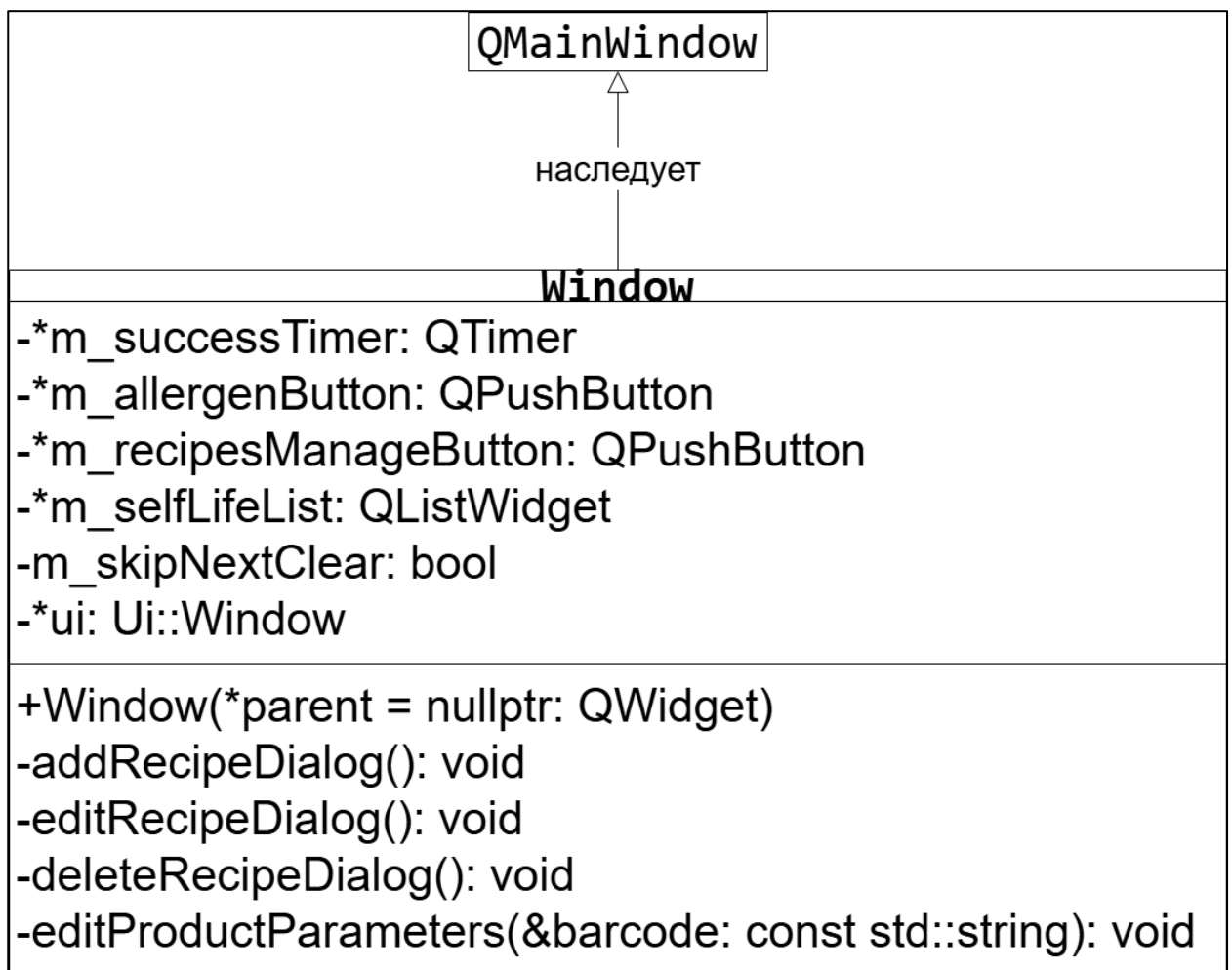


Рисунок 3 - UML диаграмма класса Window

1.2.1.3 Класс Window

Класс Window реализует графический интерфейс программы, обрабатывает ввод штрихкода, управляет отображением списков доступных рецептов, продуктов-аллергенов и просроченных продуктов. Соединяет сигналы от FixedLineEdit и кнопок с функциями модулей prods.cpp и recipes.cpp. UML диаграмма класса представлена на рисунке 3.

1.2.2 Модуль prods.cpp

Данный модуль отвечает за операции с продуктами: добавление, учёт количества, контроль аллергенов, отслеживание сроков годности после вскрытия. Все данные хранятся в файле prods.json, пример структуры файла изображён на рисунке 4. Каждый продукт идентифицируется штрихкодом.

```
{
  "4620006673396": {
    "Аллергены": "лактоза",
    "Дата вскрытия": "2026-05-20", # Если включено отслеживание срока годности
    "Количество": 1,
    "Марка": "Маслозавод нытвенский",
    "Название": "Молоко",
    "Общий объём": 600,
    "Объём": 800,
    "Отслеживать": true,
    "Срок годности после вскрытия": 5 # Если включено отслеживание срока годности
  }
}
```

Рисунок 4 – Структура файла prods.json

Функции add_prod / enter_prod отвечают за добавление продукта: автоматический запрос к API (Open Food Facts) или ручной ввод. Записываются название, марка, объём, аллергены, срок годности после вскрытия.

Функция reduce_prod выполняет уменьшение общего объёма продукта. Вычисляет, сколько целых единиц списать, корректирует Количество и Общий объём.

Функция update_product_expiry_tracking контролирует срока годности после вскрытия: если общий объём не кратен объёму единицы (упаковка открыта) – добавляет дату открытия; если кратен – сбрасывает отслеживание.

Функция `check_product_allergens` сравнивает аллергены продукта с пользовательским списком (из `allergens.json`). При совпадении возвращает `true` (выводится предупреждение).

Структура файла `prods.h` приведена на рисунке 5.

```
bool is_in(std::string barcode);
void enter_prod(std::string barcode);
void add_prod(std::string barcode);
void increase_prod(std::string barcode);
void reduce_prod(std::string barcode, int volume);
void del_prod(std::string barcode);
void check_recipes_avail(std::string prod);
int get_product_count(const std::string& barcode);
int get_product_volume_per_unit(const std::string& barcode);
int get_total_volume(const std::string& barcode);
bool is_product_name_exists(const std::string& productName);
void update_product_expiry_tracking(const std::string& barcode);
bool is_product_expired(const std::string& barcode);
void discard_expired_product(const std::string& barcode);
void save_allergens(const QStringList &allergens);
void add_allergen(const QString &allergen);
void remove_allergen(const QString &allergen);
bool check_product_allergens(const QString &productAllergensStr);
void show_allergens_dialog(QWidget *parent = nullptr);
QStringList get_expired_products();
QStringList load_allergens();
```

Рисунок 5 – Структура файла `prods.h`

1.2.3 Модуль `recipes.cpp`

Модуль `recipes.cpp` обеспечивает создание, редактирование, удаление рецептов и автоматическое определение их доступности на основе текущих запасов продуктов. Данные хранятся в файле `recipes.json`. Пример структуры файла `recipes.cpp` приведён на рисунке 6.

Функция `update_product_in_recipes` - основная функция пересчёта. Вызывается после изменения продукта: обновляет флаги «Наличие» для ингредиентов во всех рецептах и пересчитывает общую «Доступность» рецепта.

Функция `recalculate_recipe` выполняет полный пересчёт одного рецепта по текущим запасам продуктов (используется после ручного редактирования рецепта).

Функции `add_recipe` / `del_recipe` отвечают созданию нового рецепта с указанием ингредиентов и их количества; удаление рецепта из `recipes.json`.

Структуру файла `recipes.h` изображена на рисунке 7.

```
{
  "Яичница": {
    "Доступность": false,
    "Ингредиенты": {
      "Яйцо": {
        "Количество": 3,
        "Наличие": false
      },
      "Хлеб": {
        "Количество": 100,
        "Наличие": true
      },
      "Соль": {
        "Количество": 3,
        "Наличие": true
      }
    }
  }
}
```

Рисунок 6 – Структура файла `recipes.json`

```
bool is_exists(std::string path);
void add_recipe(std::string name, std::map<std::string, int> products);
void add_recipe_ingr(std::string name, std::string ingr, int cnt);
void update_recipe_name(std::string name, std::string new_name);
void update_recipe_ingr(std::string name, std::string ingr, std::string new_ingr);
void update_recipe_ingr_cnt(std::string name, std::string ingr, int cnt);
void update_recipe_avail(std::string name);
void update_product_in_recipes(const std::string& productName);
void recalculate_recipe(const std::string& recipeName);
void del_recipe(std::string name);
void del_recipe_ingr(std::string name, std::string ingr);
bool iequals(const std::string& a, const std::string& b);
```

Рисунок 7 – Структура файла `recipes.h`

1.2.4 Модуль `window.cpp`

Этот модуль отвечает за визуальную часть программы. Ниже приведены основные функции.

Функция `onValidationResult` – обработка введенного штрихкода: добавление, уменьшение объёма или изменение параметров продукта.

Функция `updateRecipesList` отображает в списке только те рецепты, у которых «Доступность» == `true`.

Функция `updateAllergensList` показывает продукты, содержащие аллергены из пользовательского списка.

2 ИСПОЛЬЗУЕМЫЕ ИНСТРУМЕНТЫ

2.1 Используемые библиотеки

2.1.1 Фреймворк Qt

Qt (версия 5.12) – кросс-платформенный фреймворк для разработки GUI-приложений [2]. В проекте были использованы следующие модули:

- QtWidgets для создания главного окна, списков, диалогов;
- QtCore для работы с датами и таймерами;
- QtGui для валидации ввода;
- QtNetwork для выполнения синхронных запросов к API Open Food Facts [3];

Qt выбран благодаря удобству разработки интерфейса, наличию встроенных компонентов для работы с сетью и датами, а также кроссплатформенности.

2.1.2 Библиотека `nlohmann/json`

Данная библиотека (JSON for Modern C++ [4]) используется для сериализации и десериализации данных в формате JSON [5]. С её помощью программа сохраняет и загружает файлы. Библиотека выбрана из-за простоты синтаксиса и отсутствия необходимости в дополнительной сборке.

2.1.3 Стандартная библиотека C++

Стандартная библиотека C++ (STL) [6] применяется для базовых операций: работа с файловыми потоками (`ifstream`, `ofstream`), хранение пар «ингредиент – количество» в `std::map`, обработка строк (`std::string`) и выполнение алгоритмов (`std::min`, `std::sort`).

2.2 Другие инструменты

Также при разработке проекта использовались следующие инструменты:

- среда разработки Qt Creator 5.12;
- внешний API Open Food Facts;

- OBS Studio – программа для записи экрана.

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы разработано АРМ специалиста по учёту продуктов питания. Цель работы достигнута полностью.

Для достижения цели были решены следующие задачи:

- Выбран и освоен фреймворк Qt для разработки графического интерфейса, библиотека nlohmann/json для работы с JSON, а также публичное API Open Food Facts для автоматического получения информации о продуктах по штрихкоду.
- Разработаны функции добавления продуктов, увеличения и уменьшения их количества, полного удаления.
- Реализован механизм контроля сроков годности после вскрытия упаковки.
- Создана система управления аллергенами.
- Разработаны функции для работы с рецептами.
- Реализован графический интерфейс на Qt, обеспечивающий удобное взаимодействие через списки, диалоги и поле ввода штрихкода с валидацией.

В результате создано полнофункциональное приложение, записано и опубликовано видео работы программы, написан отчёт.

Программа протестирована на нескольких наборах тестовых данных. Все основные сценарии (добавление продукта, расход, просрочка, аллергены, приготовление блюда) работают корректно. Пользовательский интерфейс интуитивно понятен и не требует специального обучения.

Перспективы дальнейшего развития:

- добавление асинхронных запросов к API (чтобы не блокировать интерфейс);
- поддержка сканера штрихкодов через USB-подключение;
- экспорт/импорт данных в CSV или Excel;

- синхронизация между несколькими устройствами через облачное хранилище;
- возможность указывать разные единицы измерения (литры, штуки, граммы) с автоматическим пересчётом.

Таким образом, поставленная задача выполнена в полном объёме. Разработанное АРМ может быть использовано для домашнего учёта продуктов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) The Future of Digital Experiences // Qt documentation URL: <https://doc.qt.io/> (дата обращения: 22.05.26).
- 2) GUI // SkillFactory Media URL: <https://blog.skillfactory.ru/glossary/gui/> (дата обращения: 22.05.26).
- 3) API // OpenFoodFacts URL: <https://openfoodfacts.github.io/openfoodfacts-server/api/> (дата обращения: 22.05.26).
- 4) API Documentation // JSON for Modern C++ URL: https://json.nlohmann.me/api/basic_json/ (дата обращения: 22.05.26).
- 5) Введение в JSON // JSON URL: <https://www.json.org/json-ru.html> (дата обращения: 22.05.26).
- 6) STL: стандартная библиотека шаблонов C++ // Яндекс Практикум URL: <https://practicum.yandex.ru/blog/biblioteka-standartnyh-shablonov-stl/#konteynery> (дата обращения: 22.05.26).